LEGION2 Development Guide

Overview

LEGION2 implements a unified streaming pipeline architecture built on three core traits: `Source`, `Transform`, and `Sink`. Data flows through the system as `Observation` objects, processed by an `Engine` orchestrator that works with a `Registry` to wire components dynamically.

```rust
engine_execute(plan) → streaming pipeline → real-time results
```

Architecture Deep Dive

Core Data Flow

```
User Input (Frontend)
  ↓
engine_execute(Plan)
  ↓
Registry (Component Factory)
  ↓
Engine Orchestrator
  ├→ Source (Scanner)
  │   ├→ MasscanScanner: Fast port discovery
  │   └→ NmapScanner: Detailed service scanning
  ↓
  ├→ Transform Pipeline
  │   ├→ IpEnrichmentTransform: Extract IPs from raw output
  │   ├→ ServiceParsingTransform: Parse service details
  │   └→ ProgressTrackingTransform: Monitor scan progress
  ↓
  └→ Broadcast (Parallel Distribution)
      ├→ UiSink: Real-time UI updates via Tauri events
      ├→ DbSink: Persistent storage in encrypted SQLite
      └→ VulnerabilityAnalysisSink: Security analysis
```

The Observation Object

The `Observation` is the fundamental data unit flowing through the pipeline:

```rust
struct Observation {
    scan_id: Uuid,
    kind: ObservationKind,  // Host, Service, Banner, Metric, Error
    fields: Map<String, Value>,  // Flexible data storage
    ts: DateTime,
    key: String,  // Unique identifier
    raw: Option<String>,  // Original scanner output
}
```

Component Architecture

Sources (Data Producers)

Sources spawn external processes (nmap/masscan) and convert their output into Observations:

- **MasscanScanner**: Optimized for speed, discovers open ports quickly
- **NmapScanner**: Detailed scanning with service detection, OS fingerprinting
- **Future**: Rustscan, Zmap, custom scripts

Transforms (Data Processors)

Transforms enrich and parse raw observations:

- **IpEnrichmentTransform**: Extracts IP addresses from scanner output
- **ServiceParsingTransform**: Identifies services, versions, banners
- **ProgressTrackingTransform**: Calculates scan completion percentage
- **Future**: CVE mapping, correlation engine, threat intelligence

Sinks (Data Consumers)

Sinks handle processed observations:

- **UiSink**: Emits Tauri events (`obs:host`, `obs:service`, `obs:vulnerability`)
- **DbSink**: Stores in SQLite with foreign keys and indexes
- **VulnerabilityAnalysisSink**: Analyzes services for security issues

Registry Pattern

The Registry implements a plugin architecture for extensibility:

```rust
Registry {
    sources: HashMap<String, Box<dyn Source>>,
    transforms: HashMap<String, Box<dyn Transform>>,
    sinks: HashMap<String, Box<dyn Sink>>,
}
```

Benefits:

- **Dynamic Pipeline Construction**: Configure scan pipelines at runtime
- **Plugin Support**: Add new components without modifying core code
- **Testing**: Mock components for unit tests
- **Configuration-Driven**: Define pipelines in JSON/YAML

Repository Structure

Backend (`src-tauri/`)

```
src-tauri/
├── main.rs              # Tauri initialization, dependency injection
├── database.rs          # Encrypted SQLite wrapper (rusqlite only)
├── plan.rs              # Plan builder for engine_execute
├── core/
│   ├── engine.rs        # Pipeline orchestrator with broadcaster
│   ├── registry.rs      # Component factory and registration
│   ├── traits.rs        # Source, Transform, Sink interfaces
│   ├── transforms.rs    # Data processing components
│   └── sinks.rs         # UI, DB, Vulnerability sinks
├── scanning/
│   ├── masscan.rs       # Masscan source implementation
│   └── nmap.rs          # Nmap source with stateful parser
├── analysis/
│   ├── engine.rs        # Analysis orchestrator
│   └── vulnerability.rs # CVE detection, CVSS scoring
├── commands/
│   ├── engine_commands.rs # engine_execute Tauri command
│   └── plan_commands.rs  # Plan building utilities
├── utils/
│   ├── parsing.rs       # Stateful NmapParser for context tracking
│   ├── network.rs       # IP validation, broadcast handling
│   └── os.rs            # Binary path detection
└── .logs/
    └── .logs.log        # Runtime logs for debugging
```

Frontend (`src/`)

```
src/
├── App.tsx              # Root component mounting ScannerPanel
├── components/
│   ├── ScannerPanel.tsx  # Main dashboard orchestrator
│   ├── ScanForm.tsx      # Scan configuration UI
│   ├── HostTable.tsx     # Host discovery display
│   ├── ResultViewer.tsx  # Port/service/vulnerability viewer
│   └── NetworkMap.tsx    # Network topology visualization
├── stores/
│   ├── appStore.ts       # Minimal event listener (backend-driven)
│   └── hostStore.ts      # Host state management
└── services/
    └── tauriApi.ts       # Backend communication layer
```

Data Flow Implementation

1. Scan Initiation

typescript

```typescript
// Frontend
const plan = {
   scan_id: uuid(),
   targets: "192.168.1.0/24",
   ports: "1-1000",
   source_type: "nmap",
   modules: ["ip_enrichment", "service_parsing"],
   sink_types: ["ui", "db", "vulnerability"]
};
await invoke('engine_execute', { plan });
```

2. Backend Processing

rust

```rust
// Engine execution
let source = registry.create_source(&plan)?;      // Create scanner
let stream = source.start(&plan)?;                // Start scanning
let transformed = transform.apply(stream)?;       // Process data
let sinks = registry.create_sinks(&plan)?;        // Create outputs
broadcaster.distribute(transformed, sinks).await?;  // Parallel distribution
```

3. Real-time Events

rust

```rust
// UiSink emits Tauri events
app.emit("obs:host", {
    ip: "192.168.1.100",
    hostname: "server.local",
    status: "up",
    mac_address: "00:11:22:33:44:55",
    vendor: "Dell Inc."
});

app.emit("obs:service", {
    ip: "192.168.1.100",
    port: 22,
    protocol: "tcp",
    service: "ssh",
    version: "OpenSSH 7.4"
});
```

4. Frontend Reception

typescript

```typescript
// appStore listens to backend events
await listen('obs:host', (event) => {
    updateHosts(event.payload);
});

await listen('obs:service', (event) => {
    updateServices(event.payload);
});
```

Critical Issues & Solutions

Issue 1: Ports Not Displaying

Problem: Services detected but not shown in UI **Root Cause**: DbSink stores services but frontend queries fail **Solution**:

```rust
// In DbSink::store_service_detailed
self.db.upsert_service_detailed(
    ip, port, protocol, state,
    service, version, banner
).await?;

// Ensure host exists before storing service
self.db.upsert_host(ip, None, Some("up")).await?;
```

Issue 2: Scan Stuck After Completion

Problem: "Scan completed" blocks new scans **Root Cause**: Engine not properly cleaning up resources **Solution**:

```rust
// In engine.rs
impl Engine {
    pub async fn reset(&mut self) {
        self.registry.clear_active_sources();
        // Reset all stateful components
    }
}
```

Issue 3: Missing Host Details

Problem: MAC, OS, vendor info showing "Unknown" **Root Cause**: NmapParser not extracting all fields **Solution**:

```rust
// Enhanced NmapParser in utils/parsing.rs
impl NmapParser {
    fn parse_mac_address(&self, line: &str) -> Option<MacInfo> {
        // Parse "MAC Address: XX:XX:XX:XX:XX:XX (Vendor)"
        let regex = Regex::new(r"MAC Address:\s+([0-9A-Fa-f:]+)(?:\s+\(([^)]+)\))?").unwrap();
        // Extract and return MacInfo
    }

    fn parse_os_detection(&self, line: &str) -> Option<OsInfo> {
        // Parse "OS details: Linux 3.2 - 4.9"
        // Extract OS family, version, accuracy
    }
}
```

Issue 4: Network Barrier (10.0.0.0/24)

Problem: Cannot scan beyond local subnet **Root Cause**: Firewall or routing configuration **Solution**:

1. Check Windows Firewall rules for nmap/masscan
2. Run with elevated privileges
3. Add explicit routing:

```rust
// In masscan/nmap command building
if target.starts_with("10.") {
    cmd.arg("--privileged");
}
```

Issue 5: Vulnerability Display

Problem: Vulnerabilities detected but not shown **Root Cause**: Frontend not properly handling vulnerability events **Solution**:

```typescript
// In ResultViewer.tsx
useEffect(() => {
    const loadVulnerabilities = async () => {
        if (currentHost) {
            const vulns = await invoke('get_host_vulnerabilities', {
                hostId: currentHost.id
            });
            setHostVulnerabilities(vulns);
        }
    };
    loadVulnerabilities();
}, [currentHost]);
```

Performance Optimizations

Parallel Processing

```rust
// Engine broadcasts to all sinks concurrently
let tasks = sinks.map(|sink| {
    tokio::spawn(sink.run(obs_stream.clone()))
});
futures::future::join_all(tasks).await;
```

Database Write Batching

```rust
// DbSink uses spawn_blocking for heavy writes
tokio::task::spawn_blocking(move || {
    db.batch_insert_services(services)
}).await?;
```

Stream Buffering

```rust
// Use bounded channels to prevent memory overflow
let (tx, rx) = broadcast::channel::<Observation>(1024);
```

Extension Guide

Adding a New Scanner

1. Implement the `Source` trait:

```rust
pub struct RustscanSource;

#[async_trait]
impl Source for RustscanSource {
    fn name(&self) -> &'static str { "rustscan" }
    async fn start(&self, plan: &Plan) -> Result<ObsStream> {
        // Spawn rustscan process
        // Parse output into Observations
        // Return stream
    }
}
```

2. Register in Registry:

```rust
registry.register_source("rustscan", Box::new(RustscanSource));
```

3. Use in plans:

```json
{
    "source_type": "rustscan",
    "targets": "192.168.1.0/24"
}
```

Adding a Vulnerability Rule

```rust
// In vulnerability.rs
let rule = VulnerabilityRule {
    id: "CVE-2024-1234",
    name: "Critical SSH Vulnerability",
    pattern: regex::Regex::new(r"OpenSSH.*[1-6]\.").unwrap(),
    severity: Severity::Critical,
    cvss_score: Some(9.8),
};
rules.push(rule);
```

Debugging Guide

Enable Verbose Logging

```rust
// Set in main.rs
env_logger::Builder::from_env(
    env_logger::Env::default().default_filter_or("debug")
).init();
```

Check Logs

```bash
tail -f src-tauri/.logs/.logs.log
```

Common Log Patterns

```
Engine starting execution...        # Scan started
Broadcasting observation: Host      # Host discovered
Storing service 192.168.1.1:22      # Service saved
Found 3 vulnerabilities             # Analysis complete
```

Best Practices

1. **Keep Frontend Minimal**: Frontend only displays, backend processes
2. **Use Type Safety**: Leverage Rust's type system for correctness
3. **Handle Errors Gracefully**: Use Result<T> everywhere
4. **Test Components Individually**: Each trait implementation should be testable
5. **Document Observations**: Always include raw output for debugging
6. **Batch Database Operations**: Group writes for performance
7. **Use Async Properly**: Don't block the runtime with sync operations

Future Roadmap

Phase 1: Core Stability (Current)

- Fix port/service display issues
- Implement proper scan reset
- Complete vulnerability analysis

Phase 2: Advanced Features

- Attack path generation
- Exploit correlation
- Custom script support
- Report generation

Phase 3: Enterprise Features

- Multi-user support
- Distributed scanning
- API gateway
- Cloud integration

Contributing

1. Follow the pipeline architecture - new features as Sources, Transforms, or Sinks
2. Maintain the single `engine_execute` command interface
3. Keep the `Db` wrapper as the sole database access point
4. Write comprehensive tests for new components
5. Document all Observation types and fields
6. Use the established logging patterns

Quick Reference

Key Commands

```bash
# Backend tests
cd src-tauri && cargo test

# Frontend tests
npm test

# Build for production
npm run tauri build

# Development mode
npm run tauri dev
```

Architecture Principles

- **Single Responsibility**: Each component does one thing well
- **Stream Processing**: Data flows continuously, no blocking
- **Event-Driven**: UI reacts to backend events
- **Plugin Architecture**: Extensible without core modifications
- **Type Safety**: Rust and TypeScript provide compile-time guarantees

Support

- Logs: `src-tauri/.logs/.logs.log`
- CLI docs: `docs/massnmap.md`
- Issues: Check Registry initialization, Database connection, Event listeners